

LifeTrack Backend

Complete Interview Question Bank

180 Questions · 14 Topics · ASP.NET Core · EF Core · JWT · Microservices

Every backend topic behind the LifeTrack clinical trial platform

C# Fundamentals · ASP.NET Core · Entity Framework Core
JWT Security · DI & Filters · Microservices Architecture
Caching · REST Design · Design Patterns · SQL Server
Configuration · Testing · Advanced .NET · Project Q&A

Table of Contents | # | Topic | Questions | |---|---|---| | 1 | C# Language Fundamentals | Q1 - Q20 | | 2 | ASP.NET Core (Middleware, Controllers, Routing) | Q21 - Q38 | | 3 | Entity Framework Core (Migrations, DbContext, Queries) | Q39 - Q60 | | 4 | JWT Authentication and Security | Q61 - Q78 | | 5 | Dependency Injection and Filters | Q79 - Q90 | | 6 | Microservices Architecture | Q91 - Q104 | | 7 | Caching (IMemoryCache, Version-Bump Pattern) | Q105 - Q111 | | 8 | REST API Design | Q112 - Q119 | | 9 | Architecture and Design Patterns | Q120 - Q135 | | 10 | SQL Server and Database Design | Q136 - Q149 | | 11 | Configuration, Swagger, and Tooling | Q150 - Q155 | | 12 | Testing Concepts | Q156 - Q160 | | 13 | Advanced .NET and C# | Q161 - Q170 | | 14 | Project-Specific Deep Dives | Q171 - Q180 |

Topic 1: C# Language Fundamentals

1.1 OOP Concepts

Q1.

What is the difference between a `class` and a `struct` in C#? When would you use a struct over a class? **A:** Classes are reference types allocated on the heap; structs are value types allocated on the stack. Use structs for small, immutable data (like coordinates or money) where copying is cheap. In LifeTrack, all entities (`User`, `Protocol`, `PatientRecord`) are classes because they are mutable, large, and need reference semantics through EF Core's change tracker.

Q2.

Explain the four pillars of OOP with examples from your project. **A:** **Encapsulation** — `BCryptPasswordHasher` hides the hashing algorithm inside `Hash()` and `Verify()` methods; callers never see the BCrypt internals. **Inheritance** — all controllers inherit from `ControllerBase`, getting `Ok()`, `NotFound()`, `BadRequest()` methods. **Polymorphism** — `IUserRepository` interface allows `UserRepository` and a future mock to be swapped without changing `UserService`. **Abstraction** — `IAuthService` defines what auth does (`LoginAsync`, `RegisterAsync`) without exposing how.

Q3.

What is the difference between an abstract class and an interface? When do you use each? **A:** Abstract classes can have implementation, fields, and constructors; they represent an "is-a" relationship. Interfaces define a contract with no implementation (until default interface methods in C# 8+); they represent "can-do". In LifeTrack, `IUserRepository` and `IAuthService` are interfaces because multiple implementations are possible (real DB, mock for tests). If you had shared logic across all repositories, you'd use an abstract base class.

Q4.

What are `sealed` classes? Where does your project use them? **A:** `sealed` prevents class inheritance. `DomainException` and `NotFoundException` in `Shared.CL` are `sealed` — you don't want developers subclassing error types, which could bypass the `DomainExceptionFilter` pattern. `ErrorDetail` is also sealed.

Q5.

Explain the `new` keyword hiding vs `override` virtual method. Where is it used in your project? **A:** `override` replaces a virtual base method polymorphically — calling through a base reference still executes the derived version. `new` hides the base method — calling through a base reference executes the base version. Your `AuthDbContext.SaveChangesAsync()` uses `public new async Task<int> SaveChangesAsync()` — hiding the base `DbContext.SaveChangesAsync()`. This means callers who hold an `AuthDbContext` reference get the audit version; callers holding a `DbContext` reference get the base. For the auto-audit to work properly everywhere, it would ideally be `override` — the `new` approach is a design choice.

1.2 C# Records and DTOs

Q6.

What is a C# `record` and how does it differ from a `class`? Why did you use records for DTOs? **A:** Records are reference types (C# 9+) with value-based equality, immutability by default, built-in `ToString()`, and concise positional syntax. `CreateAdverseEventRequest`, `PatientResponse`, all DTO types in your project are records. Why: DTOs are read-only input/output — you create them, read them, and discard them. Immutability prevents accidental mutation after model binding. Positional syntax like `new(x.EventID, x.PatientID, ...)` is concise.

Q7.

What is the `with` expression in C# records? Where does your project use it? **A:** `with` creates a new record with only the specified properties changed — non-destructive mutation. In `PatientsController.Create()`:

```
finalReq = req with { UserID = callerId };
```

This creates a new `CreatePatientRequest` identical to `req` except `UserID` is overridden to the JWT caller's ID. Original `req` is unchanged. This is the IDOR protection pattern.

Q8.

What is the `??` null-coalescing operator and `?.` null-conditional operator? Give examples from your project. **A:** `??` returns the left side if not null, otherwise the right side. In `JwtAuthFilter`: `Environment.GetEnvironmentVariable("JWT_SECRET") ?? config["Jwt:Secret"]` — prefers environment variable, falls back to config. `?.` short-circuits to null if the left side is null. In `AuthDbContext`: `_httpCtx?.HttpContext?.User?.FindFirstValue(...)` — if any part of the chain is null, returns null instead of

throwing `NullReferenceException`.

Q9.

What is the `??=` null-coalescing assignment operator? **A:** `x ??= defaultValue` assigns `defaultValue` to `x` only if `x` is currently null. Not used in LifeTrack but common in lazy initialization: `_instance ??= new Service()`.

Q10.

Explain `is`, `as`, and pattern matching with `is type varName`. Where are they used? **A:** `is` checks type at runtime. `as` safely casts (returns null if fails). Pattern matching: `if (ex is DomainException domEx) in DomainExceptionFilter` — checks if `ex` is a `DomainException` AND binds it to `domEx` in one expression. The switch expression `entry.State switch { EntityState.Added => "Insert", ... }` in `SaveChangesAsync` is also pattern matching.

1.3 LINQ and Collections

Q11.

What is deferred execution in LINQ? How does it apply to your EF Core queries? **A:** LINQ queries are not executed when defined — they're executed when iterated (`ToList()`, `CountAsync()`, `FirstOrDefault()`). In your repositories:

```
var q = _db.Protocols.AsQueryable(); // no SQL yet
if (!string.IsNullOrEmpty(status))
    q = q.Where(p => p.Status == status); // still no SQL
var total = await q.CountAsync(); // SQL runs: SELECT COUNT(*)
var items = await q.ToListAsync(); // SQL runs: SELECT * WITH pagination
```

This is why you can chain `.Where()` conditionally and still get one optimized SQL query.

Q12.

What is the difference between `IEnumerable<T>`, `IQueryable<T>`, and `IReadOnlyList<T>`? **A:** `IEnumerable<T>` — in-memory iteration, LINQ runs in C#. `IQueryable<T>` — query expression that translates to SQL; EF Core uses this so `.Where()` becomes `WHERE` in SQL. `IReadOnlyList<T>` — in-memory, indexed, read-only. Your repositories return `IReadOnlyList<T>` to prevent callers from modifying the result. Service layer works with `IQueryable` via EF Core.

Q13.

Explain `Select()`, `Where()`, `SelectMany()`, `Any()`, `All()`, `FirstOrDefault()` with project examples. **A:** `Where()` — `q.Where(u => u.IsActive == true)` in `UserRepository.ListAsync()`. `Select()` — `.Select(e => e.ErrorMessage)` in `ValidateModelFilter` to extract error messages. `SelectMany()` — `context.ModelState.Values.SelectMany(v => v.Errors)` flattens nested collections. `Any()` — `await db.Roles.AnyAsync()` in `DbSeeder` to check if roles already seeded. `FirstOrDefault()` — `entry.Properties.FirstOrDefault(p => p.Metadata.IsPrimaryKey())` to get PK property.

Q14.

What is `ToHashSet()` and why is it used in `DerivePatientStatus`? **A:** `ToHashSet()` creates a `HashSet<T>` from a collection, enabling `O(1)` `.Contains()` lookups instead of `O(n)` on a list. In `EnrollmentService.DerivePatientStatus()`:

```
var statuses = enrollments.Select(e => e.Status)
                        .ToHashSet(StringComparer.OrdinalIgnoreCase);
if (statuses.Contains("Active")) return "Active";
```

With `OrdinalIgnoreCase`, `"active"` and `"Active"` are treated the same. `O(1)` per check regardless of how many enrollments exist.

Q15.

What is `AsNoTracking()` and when should you use it? **A:** Tells EF Core not to track returned entities in the change tracker — saving memory and CPU. Use for read-only queries where you won't call `SaveChanges()`. In `PatientRepository.ListAsync()`: `_db.Patients.AsNoTracking()` because list results are never mutated. For `GetByIdAsync()` (which is followed by updates), tracking is needed — no `AsNoTracking()`.

1.4 Async/Await

Q16.

What is the difference between `Task`, `Task<T>`, and `ValueTask<T>`? **A:** `Task` — represents an asynchronous operation with no return value (like `void`). `Task<T>` — async operation that returns `T`. `ValueTask<T>` — performance-oriented version that avoids heap allocation for frequently synchronous paths. Your project uses `Task` and `Task<T>` everywhere. `ValueTask` would be an optimization for hot paths.

Q17.

What does `async` / `await` actually do under the hood? **A:** The C# compiler transforms an `async` method into a state machine. Each `await` point is a state boundary — when the awaited task completes, execution resumes from that state. The calling thread is not blocked; it can process other requests. In `AuthService.LoginAsync()`, awaiting `_users.GetByEmailAsync(email)` releases the thread to handle other HTTP requests while the DB query runs.

Q18.

What is `ConfigureAwait(false)` and why is it sometimes used? **A:** By default, `await` resumes on the original synchronization context (UI thread or ASP.NET request context). `ConfigureAwait(false)` says "don't capture the sync context — resume on any thread pool thread." In library code (like `Shared.CL`) this prevents deadlocks and improves performance. Your project doesn't use it because ASP.NET Core doesn't have a synchronization context issue, but library authors should.

Q19.

What happens if you call an async method without `await`? **A:** The call returns a `Task` immediately without waiting. The operation runs in the background and any exception is swallowed unless you `.Wait()` or `await` it later. In `AuthService`, `await _audit.RecordAsync(...)` is always awaited to ensure the audit log is written before the method returns.

Q20.

What is `Task.WhenAll()` vs `Task.WhenAny()`? **A:** `WhenAll` — waits for ALL tasks to complete, returns array of results. `WhenAny` — returns when the FIRST task completes. Example: if your ProtocolSite service needed to validate both site AND protocol existence concurrently: `await Task.WhenAll(siteTask, protocolTask)` instead of two sequential awaits. Your project uses sequential validation in the 5-step chain — parallel would be faster but more complex.

Topic 2: ASP.NET Core

2.1 Middleware and Pipeline

Q21.

What is the ASP.NET Core middleware pipeline? How is it different from filters? **A:** Middleware sits at the HTTP pipeline level — it processes every request regardless of routing. Filters sit at the MVC level — they only apply to controller actions. Order: Middleware runs first (GlobalExceptionHandlerMiddleware → CORS → routing), then filters run (JwtAuthFilter → ValidateModelFilter → action → DomainExceptionHandlerMiddleware → ActivityLogFilter). In your project, GlobalExceptionHandlerMiddleware is middleware (registered with app.UseMiddleware<>()), while JwtAuthFilter is a filter (registered with o.Filters.Add<>()).

Q22.

Explain the RequestDelegate pattern in middleware. How does your GlobalExceptionHandlerMiddleware implement it? **A:** RequestDelegate is Func<HttpContext, Task> — a function that processes an HTTP context. Middleware chains these: each middleware calls _next(context) to pass to the next one. In GlobalExceptionHandlerMiddleware:

```
public async Task InvokeAsync(HttpContext context) {  
    try { await _next(context); } // calls the next middleware in chain  
    catch (Exception ex) { await HandleExceptionAsync(context, ex); }  
}
```

If no exception: _next runs the entire rest of the pipeline (routing, filters, controller). If any part throws: the catch block handles it.

Q23.

What is the order of Use, Map, Run in middleware registration? What happens if you reverse UseCors and UseOcelot? **A:** Use — adds middleware that calls _next. Map — branches the pipeline on a path prefix. Run — terminal middleware (no _next). If you put app.UseOcelot() before app.UseCors(), Ocelot forwards the request downstream before CORS headers are added to the response — the browser receives the response without Access-Control-Allow-Origin and rejects it. Order matters critically.

Q24.

What is WebApplication.CreateBuilder() and what does it do behind the scenes? **A:** Creates a WebApplicationBuilder that pre-configures: logging (console, debug), configuration sources (appsettings.json, environment variables, command line), DI container, and Kestrel (the embedded web server). Replaced CreateDefaultBuilder() + ConfigureWebHostDefaults() from .NET 5.

Q25.

What is the difference between app.UseAuthentication() and app.UseAuthorization()? Does your project use them? **A:** UseAuthentication reads and validates credentials (JWT, cookies, etc.) and sets HttpContext.User. UseAuthorization checks if the authenticated user has permission to access the resource. Your project does **not** use either — because both are replaced by JwtAuthFilter from Shared.CL. The filter handles validation and population of HttpContext.User itself. This is an intentional architectural choice — centralized custom auth logic.

2.2 Controllers and Action Results

Q26.

What is the difference between Controller and ControllerBase? Which does your project use and why? **A:** Controller inherits from ControllerBase and adds View-related methods (View(), PartialView(), etc.) for Razor Pages/MVC. ControllerBase is for API controllers only. All your controllers use ControllerBase — this is a pure API project with no UI rendering. Using Controller would add unnecessary overhead.

Q27.

What does [ApiController] attribute do? What behaviors does it enable? **A:** Three key behaviors: (1) Automatic model validation — if ModelState.IsValid is false, returns 400 automatically without you writing the check (though in your project ValidateModelFilter does this explicitly). (2) [FromBody] is inferred on complex type parameters. (3) Better error response formatting. All your controllers have [ApiController].

Q28.

What is the difference between Ok(), CreatedAtAction(), NoContent(), NotFound(), BadRequest() return types? **A:** Ok(data) → HTTP 200 with body. CreatedAtAction(action, routeValues, body) → HTTP 201 with Location header pointing to the new

resource's URL. `NoContent()` → HTTP 204, no body (used in `EnrollmentsController.Update`). `NotFound()` → HTTP 404. `BadRequest(obj)` → HTTP 400. Your project uses all of these consistently: creates use `CreatedAtAction`, updates use `Ok` (full body) or `NoContent`, individual GETs use `NotFound()` when null.

Q29.

What does `ActionResult` vs `ActionResult<T>` mean? **A:** `ActionResult` — can return any HTTP result but the return type is unknown to Swagger/OpenAPI. `ActionResult<T>` — allows returning either an HTTP result (`NotFound()`, `BadRequest()`) OR a typed value `T` (EF Core model, DTO). Swagger can read the `T` type and generate proper documentation. Most of your controllers return `ActionResult<PatientResponse>`, `ActionResult<PagedResult<T>>` etc. for proper API documentation.

Q30.

What is `[HttpGet("{id:long}")]` — what does `:long` mean? **A:** It's a **route constraint**. `:long` means the `{id}` segment is only matched if it can be parsed as a `long`. If someone calls `/api/patients/abc`, this route doesn't match and ASP.NET returns 404 before your code even runs. Without the constraint, `abc` would reach your action and cause a runtime binding failure. Common constraints: `:int`, `:long`, `:guid`, `:bool`, `:datetime`, `:minlength(3)`.

2.3 Model Binding and Validation

Q31.

What is model binding? What is the binding source order when no attribute is specified? **A:** Model binding is the automatic mapping of HTTP data to action parameters. Default source order: (1) Form values, (2) Route values, (3) Query string. For complex types (classes/records), ASP.NET infers `[FromBody]`. For simple types that match route templates, it infers `[FromRoute]`. This is why you can write `Get(long id)` without `[FromRoute]` when `{id}` is in the route template.

Q32.

What data annotations do you use for validation? How does `ModelState` work? **A:** `[Required]` — property must not be null/empty. `[MaxLength(n)]` — string length limit. `[EmailAddress]` — format validation. `[Range(min, max)]` — numeric range. ASP.NET Core's model binder evaluates these after binding. Any violation sets `ModelState.IsValid = false` and adds errors to `ModelState`. Your `ValidateModelFilter` reads `ModelState.Values.SelectMany(v => v.Errors)` and returns 422.

Q33.

What is the difference between `[Required]` and non-nullable properties in C# with nullable enabled? **A:** Nullable enabled (`<Nullable>enable</Nullable>` in `.csproj`) means the compiler warns if you assign null to a non-nullable property. But that's compile-time only — at runtime, JSON deserialization can still produce null for non-nullable types. `[Required]` is a runtime validation attribute — it catches null values from incoming JSON and adds a `ModelState` error. Your DTOs use records with `[Required]` on mandatory fields to get both compile-time and runtime safety.

Q34.

What does `[FromBody]`, `[FromQuery]`, `[FromRoute]`, `[FromForm]`, `[FromHeader]` bind from? **A:** `[FromBody]` — HTTP request body (JSON deserialization). `[FromQuery]` — URL query string `?key=value`. `[FromRoute]` — URL path segments `{id}`. `[FromForm]` — HTML form data / multipart (file uploads). `[FromHeader]` — HTTP request headers. `[FromServices]` — DI container directly into action method.

Q35.

When would you need `[FromServices]` on an action parameter? **A:** When a service is needed in only one action method and you don't want to inject it in the constructor (bloating the constructor for one rarely-used dependency). Example: a `IPdfExporter` needed only in one report action. Your project doesn't use this — all services are constructor-injected — which is the cleaner approach.

2.4 Routing

Q36.

What is the difference between conventional routing and attribute routing? Which does your project use? **A:** Conventional routing — defined globally `app.MapControllerRoute("default", "{controller}/{action}/{id?}")`. Attribute routing — `[Route("api/patients")]` and `[HttpGet("{id:long}")]` on controllers/actions. Your project uses **attribute routing exclusively** — all controllers have `[Route("api/...")]` and each action has `[HttpGet]`, `[HttpPost]` etc. Attribute routing is standard for APIs because it's explicit and colocated with the action.

Q37.

What happens when two routes match the same URL? How does ASP.NET resolve it? **A:** ASP.NET Core uses a routing precedence algorithm: more specific routes beat more general ones. Literal segments beat parameters, parameters beat

catch-alls. If two routes are equally specific, you get `AmbiguousMatchException` at runtime. In your Ocelot config, each resource has an exact route AND a wildcard — Ocelot resolves exact matches first.

Q38.

What is the purpose of `app.MapControllers()` vs `app.MapControllerRoute()`? **A:** `MapControllers()` — scans for controllers and enables attribute routing on them (used in API projects). `MapControllerRoute()` — sets up a conventional route template (used in MVC projects with Views). All your services use `MapControllers()` because they are REST APIs with no view rendering.

Topic 3: Entity Framework Core

3.1 Code First and Migrations

Q39.

What is Code First vs Database First in EF Core? Which does your project use? **A:** Code First — you write C# model classes and EF generates the DB schema. Database First — you have an existing DB and EF scaffolds the models. Your project is **Code First** — all entities (`Protocol` , `Patient` , `Document` etc.) are C# classes, and `dotnet ef migrations add` + `dotnet ef database update` create the tables. This is the modern standard.

Q40.

What is the difference between `EnsureCreatedAsync()` and `MigrateAsync()` ? **A:** `EnsureCreatedAsync()` — creates the database and tables directly from the current model snapshot if they don't exist. No migration history, no incremental updates, no rollback. Used in `Authentication.API` on startup for simplicity. `MigrateAsync()` — applies pending migrations from `__EFMigrationsHistory` table in order. Supports incremental schema changes and rollback. Used for production services (ProtocolSite, Patient, etc.) via `dotnet ef database update`.

Q41.

What is the `__EFMigrationsHistory` table? What does it contain? **A:** A table EF Core creates in your database with two columns: `MigrationId` (the timestamp+name like `20260525100137_InitialCreate`) and `ProductVersion`. EF Core checks this table to know which migrations have already been applied and skips them. When you fixed your DocumentCompliance.API schema drift by manually inserting migration IDs, you were directly editing this table.

Q42.

Explain the `Up()` and `Down()` methods in a migration file. What does the `.Designer.cs` file do? **A:** `Up()` — applies the migration (creates tables, adds columns, creates indexes). `Down()` — reverses it (drops tables, removes columns). EF Core calls `Up()` when running `dotnet ef database update` and `Down()` when rolling back. The `.Designer.cs` file is a snapshot of the model state *at the time the migration was created* — used by EF Core to compute diffs for the next migration.

Q43.

What is `ModelSnapshot.cs` ? Why is it important? **A:** The auto-generated snapshot of your full EF Core model at the current state. When you run `dotnet ef migrations add NewMigration`, EF Core diffs the current `DbContext.OnModelCreating()` against `ModelSnapshot.cs` to find what changed. If snapshot is out of sync with the model, you get `PendingModelChangesWarning` — exactly what happened in DocumentCompliance.API due to schema drift.

Q44.

What is `ExcludeFromMigrations()` and why did you use it? **A:** `modelBuilder.Entity<AuditEntry>().ToTable("AuditEntries", t => t.ExcludeFromMigrations())` tells EF Core: "I know about this table (I can write to it), but do not generate CREATE/ALTER/DROP for it in migrations." Used in AdverseEvent.API, Patient.API, AnalyticsReport.API, Notification.API for the shared `AuditEntries` table — each database has one `AuditEntries` table owned by one service (ProtocolSite.API for clinicalDb, Auth.API for governanceDb). All other services write to it but don't own its schema.

Q45.

What is `migrationBuilder.Sql()` and when did you use it instead of fluent migration methods? **A:** `migrationBuilder.Sql(rawSql)` executes arbitrary SQL inside a migration. Used in `AddFileColumns` (DocumentCompliance) and `AddReviewStatus` / `AddStartDateAndReviewStatus` (AnalyticsReport) because EF Core's `migrationBuilder.AddColumn()` throws if the column already exists — no idempotency guard. The raw SQL uses `IF NOT EXISTS (SELECT 1 FROM sys.columns ...)` to check first, making the migration safe to run multiple times.

3.2 DbContext and Configuration

Q46.

What is `DbContext` ? What is its role in EF Core? **A:** `DbContext` is the main class for database interaction in EF Core — it's the unit of work and the repository factory. It manages: the database connection, entity tracking (change tracker), query translation (LINQ → SQL), and schema configuration (`OnModelCreating`). Each of your services has exactly one `DbContext` (`AuthDbContext` , `PatientDbContext` , etc.).

Q47.

What is `OnModelCreating(ModelBuilder modelBuilder)` ? What can you configure there? **A:** Called once during `DbContext` initialization. Configures the EF Core model using the fluent API: table names (`ToTable`), primary keys (`HasKey`), indexes (`HasIndex`), relationships (`HasOne/WithMany`), column types (`HasColumnType`), default values (`HasDefaultValue`), delete behaviors

(`OnDelete`), and exclusions (`ExcludeFromMigrations`). Fluent API in `OnModelCreating` takes precedence over data annotations.

Q48.

What is `ValueGeneratedNever()` and where do you use it? **A:** Tells EF Core not to generate an auto-increment (`IDENTITY`) for the primary key — you will supply explicit values. Used for `Role.RoleID` in `AuthDbContext` :

```
e.Property(r => r.RoleID).ValueGeneratedNever();
```

Because `DbSeeder` inserts explicit values (1=Admin, 2=CTM, etc.). Without this, EF Core would try to generate the ID and conflict with the explicit insert.

Q49.

What is the EF Core change tracker? How does `SaveChangesAsync` use it? **A:** The change tracker monitors all entities loaded through a `DbContext`. When you call `.Find()` or `.FirstOrDefaultAsync()`, the entity enters the tracker as `Unchanged`. If you modify a property, it becomes `Modified`. If you `.Add()`, it becomes `Added`. If you `.Remove()`, it becomes `Deleted`. `SaveChangesAsync()` inspects the tracker and generates the appropriate SQL (INSERT/UPDATE/DELETE) for each tracked entity.

Q50.

What is the difference between `_db.Update(entity)` and implicit change tracking for updates? **A: Implicit** (`ProtocolSite.API` pattern): `GetByIdAsync()` loads the entity into the tracker as `Unchanged`. You mutate properties directly → tracker detects the change → entity becomes `Modified`. `SaveChangesAsync()` alone is sufficient — generates `UPDATE` only for changed columns. **Explicit** (`AdverseEvent.API` pattern): `_db.AdverseEvents.Update(ae)` forcefully marks the entire entity as `Modified` regardless of what changed — generates `UPDATE` for ALL columns. Implicit is more efficient; explicit is simpler.

Q51.

What is `AsNoTracking()` and when should you use it vs not? **A:** `AsNoTracking()` on a query tells EF Core not to add returned entities to the change tracker, saving memory and processing. Use it for **read-only queries** where you will never call `SaveChanges()`. In your list queries (`ListAsync()` methods), all results are returned to Angular without modification — perfect for `AsNoTracking()`. In `GetByIdAsync()` (which is followed by updates), you need tracking — omit `AsNoTracking()`.

Q52.

What is `FindAsync()` vs `FirstOrDefaultAsync()`? When does `FindAsync` check the cache? **A:** `FindAsync(key)` first checks the EF change tracker (in-memory) — if the entity with that key was already loaded in this request, it returns it without a DB hit. Only if not found in the tracker does it query the DB. `FirstOrDefaultAsync(predicate)` always queries the DB. Use `FindAsync` when you know the primary key. In your `DeleteAsync` methods (`var p = await _db.Patients.FindAsync(patientId)`), `FindAsync` is preferred.

3.3 Relationships and FKs

Q53.

What are the three types of EF Core relationships? Give examples from your project. **A: One-to-Many** — `Protocol` has many `SiteProtocol` entries (`HasOne/WithMany`). `Patient` has many `Enrollments`. `Enrollment` has many `Visits`. **Many-to-Many** — `SiteProtocol` IS the many-to-many junction between `Protocol` and `Site` (with extra payload fields). **One-to-One** — not used directly in your project; `PatientRecord.UserID` is a logical one-to-one but enforced via a unique filtered index, not an EF relationship.

Q54.

What is `DeleteBehavior.Restrict` vs `DeleteBehavior.Cascade`? Where did you use each? **A:** `Restrict` — attempting to delete a parent that has children throws a referential integrity error (no implicit cascade). `Cascade` — deleting a parent automatically deletes all children. In `Patient.API`: `Enrollment → Patient` uses **Restrict** (can't delete a patient with enrollments — protect clinical history). `Visit → Enrollment` uses **Cascade** (deleting an enrollment deletes its visits — a visit without an enrollment is meaningless). These are enforced at the SQL Server FK constraint level, not just EF Core logic.

Q55.

What is eager loading vs lazy loading vs explicit loading in EF Core? **A: Eager loading** — `.Include(u => u.RoleNavigation)` loads related data in the same SQL query (JOIN). Your `UserRepository.GetByIdAsync()` and `EnrollmentRepository.GetByIdAsync()` use this. **Lazy loading** — loads related data on first access of the navigation property (requires virtual properties and proxies). Not configured in your project. **Explicit loading** — `await _db.Entry(user).Reference(u => u.Role).LoadAsync()` — loads related data on demand. Not used in your project.

Q56.

What is a unique filtered index? How did you create one for `UserID` in `PatientRecord`? **A:** A unique index that only applies to

non-null rows. In `PatientDbContext.OnModelCreating()` :

```
e.HasIndex(p => p.UserID).IsUnique().HasFilter("[UserID] IS NOT NULL");
```

The SQL Server constraint: one user account maps to at most one patient record. But `UserID = null` is allowed for multiple rows (staff-created patients without accounts). Without the filter, only one null-UserID patient could exist.

Q57.

What is a cross-service reference in your project? Why no EF FK for them? **A:** Cross-service references are raw `long` IDs that point to entities owned by other services. Examples: `SiteProtocol.InvestigatorID` → Auth.API's `Users.UserID`; `AdverseEventRecord.PatientID` → Patient.API's `Patients.PatientID`. No EF FK because EF Core FKs require the referenced entity's `DbSet` to be in the same DbContext. Each service has its own DbContext pointing to its own domain. Adding the FK would require sharing the DbContext — violating service isolation.

3.4 Querying Patterns

Q58.

What is `ExecuteUpdateAsync()`? How is it different from the standard load-mutate-save pattern? **A:** `ExecuteUpdateAsync()` is an EF Core 7+ bulk update that translates directly to a single SQL `UPDATE ... SET ... WHERE` without loading any entities into memory. In `PatientRepository.UpdateEnrollmentStatusAsync()` :

```
await _db.Patients.Where(p => p.PatientID == patientId)
    .ExecuteUpdateAsync(s => s.SetProperty(p => p.EnrollmentStatus, newStatus));
```

Generates: `UPDATE Patients SET EnrollmentStatus = @newStatus WHERE PatientID = @patientId`. No change tracker involvement — safer when other entities for the same patient may already be tracked.

Q59.

What is the `Skip/Take` pattern and how does it implement pagination? **A:** `Skip((page - 1) * pageSize)` skips the rows from previous pages. `Take(pageSize)` takes only the current page's rows. Translates to SQL Server: `ORDER BY ... OFFSET x ROWS FETCH NEXT y ROWS ONLY`. The `CountAsync()` BEFORE pagination gives the total count for page navigation. Both queries run against the same filtered `IQueryable`, so the WHERE conditions are identical.

Q60.

What is projection in EF Core and why is it used in `AuditEntriesController`? **A:** Projection — using `.Select(a => new AuditEntryDto { ... })` before `.ToListAsync()`. EF Core translates this to `SELECT AuditEntryID, EntityName, ...` — only the columns needed, not `SELECT *`. Used in `AuditEntriesController` because it queries directly from DbContext without a service/repository layer. Projection also avoids tracking overhead and reduces data transferred from DB.

Topic 4: JWT Authentication and Security

4.1 JWT Fundamentals

Q61.

What does JWT stand for and what are its three parts? **A:** JSON Web Token. Three Base64url-encoded parts separated by dots: **Header** (`alg` and `typ`), **Payload** (claims), **Signature** (HMAC of header+payload using the secret). Your token: `header={"alg":"HS256","typ":"JWT"}, payload={"sub":"1","email":"admin@...", "role":"Admin", "exp":...}, signature= HMAC-SHA256(base64(header)+"."+base64(payload), secret)`.

Q62.

What is the difference between `sub`, `jti`, `iss`, `aud`, `exp`, `nbf` JWT claims? **A:** `sub` — subject (who the token is about), your `UserID`. `jti` — JWT ID, a unique identifier for this specific token (prevents replay). `iss` — issuer (`"LifeTrack"`). `aud` — audience (`"LifeTrackUsers"`). `exp` — expiration time (Unix timestamp). `nbf` — not before time (token not valid before this time). All are set in `JwtTokenService.Generate()`.

Q63.

What is HMAC-SHA256? How is the JWT signature computed? **A:** HMAC (Hash-based Message Authentication Code) using SHA-256. The signature is `HMAC-SHA256(Base64url(header) + "." + Base64url(payload), secret_key_bytes)`. The secret must be known to both the issuer (Auth.API) and all validators (all other services). If even one bit of the header or payload changes, the HMAC produces a completely different result — tampered tokens are detected.

Q64.

What is the difference between symmetric (HMAC) and asymmetric (RSA/ECDSA) JWT signing? **A:** **Symmetric (HMAC-SHA256)** — same secret key for signing and verification. Simple, fast. Security risk: every service that validates tokens must know the secret — if any service is compromised, the secret is exposed. **Asymmetric (RSA/ECDSA)** — private key signs, public key verifies. Auth.API keeps the private key secret; all other services only have the public key. More secure for distributed systems. Your project uses symmetric — simpler for a local development project. Production systems should prefer asymmetric.

Q65.

Why is JWT stateless? What does that mean for your architecture? **A:** Stateless means no server-side session store is needed. All information needed to validate the user (ID, role, email) is encoded in the token itself. Each service independently validates the token by recomputing the HMAC. This is why ProtocolSite.API can authenticate a user without calling Auth.API — everything it needs is in the token signature. Trade-off: you can't immediately invalidate a token (logout doesn't truly log out until the token expires).

Q66.

How does your `JwtAuthFilter` handle an expired token? What does `ClockSkew` do? **A:** `ValidateToken()` throws `SecurityTokenExpiredException` when `exp` < current time. `JwtAuthFilter` catches all `SecurityTokenException` subtypes and returns 401 with the exception message. `ClockSkew = TimeSpan.FromSeconds(30)` adds a 30-second grace period — a token expiring at 10:00:00 is still accepted until 10:00:30. This handles slight clock differences between machines in a distributed system.

Q67.

What claims did you include in your JWT and why each one? **A:** `JwtRegisteredClaimNames.Sub` = `UserID` — standard "who is this token for." `JwtRegisteredClaimNames.Email` = email — quick access to email without DB lookup. `JwtRegisteredClaimNames.Jti` = `Guid.NewGuid()` — unique per token, enables blacklisting if needed. `ClaimTypes.NameIdentifier` = `UserID` — ASP.NET Core convention, used by `User.FindFirstValue(ClaimTypes.NameIdentifier)` in controllers. `ClaimTypes.Name` = user's name. `ClaimTypes.Role` = role name — used by `RoleAuthorizeFilter`.

Q68.

Why did you store `UserID` in both `Sub` and `ClaimTypes.NameIdentifier`? **A:** `Sub` is the JWT standard claim name. `ClaimTypes.NameIdentifier` is ASP.NET Core's convention. Both point to the same value (`UserID`). Different parts of the codebase use different conventions: `JwtAuthFilter` reads `sub` via `JwtRegisteredClaimNames.Sub` when building `ClaimsPrincipal`. Controllers read via `ClaimTypes.NameIdentifier`. Including both makes the token work with all ASP.NET Core auth middleware and your custom code.

4.2 Security Patterns

Q69.

What is BCrypt and why is it better than MD5/SHA256 for passwords? **A:** BCrypt is a password hashing algorithm with a configurable cost factor. MD5/SHA256 are fast hash functions — an attacker with a GPU can try billions of hashes per second (brute force). BCrypt is intentionally slow — with `workFactor: 12` ($2^{12} = 4096$ rounds), each hash takes ~250ms on a modern CPU. An attacker can only try thousands per second, making brute force infeasible. BCrypt also generates a random salt per password, so two identical passwords produce different hashes (prevents rainbow table attacks).

Q70.

What is a `SaltParseException` in BCrypt and why do you catch it? **A:** Thrown by `BCrypt.Verify()` when the stored hash string is malformed or corrupted (doesn't follow the `$2a$12$...` format). Without the catch, a corrupted password hash would cause an unhandled exception and 500 response. Your `BCryptPasswordHasher.Verify()` catches it and returns `false` — treating a corrupted hash as a failed verification, which is safe behavior.

Q71.

What is user enumeration and how did you prevent it in `AuthService.LoginAsync()`? **A:** User enumeration is when an attacker can determine whether an email exists in the system based on different error messages ("email not found" vs "wrong password"). Your service returns the same message for both cases:

```
if (user is null || !_hasher.Verify(req.Password, user.PasswordHash))
    throw new AuthException("Invalid email or password.");
```

The single condition and same message prevent the attacker from learning whether the email exists.

Q72.

What is IDOR (Insecure Direct Object Reference)? How did you prevent it in `NotificationService` and `PatientsController`? **A:** IDOR is when a user can access another user's data by manipulating an ID. In `NotificationService.MarkReadAsync(id, callerUserId)`:

```
if (n.UserID != callerUserId)
    throw new UnauthorizedAccessException("You can only mark your own notifications as read.");
```

User 42 can't mark user 17's notification by guessing notification ID. In `PatientsController`, a Patient-role caller's `UserID` claim overrides any `UserID` in the request body — they can't create a patient record linked to another user's account.

Q73.

What is a compensating transaction / saga? Explain the patient registration saga in your project. **A:** A saga is a sequence of local transactions with compensating actions for rollback (since distributed transactions span multiple services). Your registration saga: (1) Create User in Auth.API/GovernanceDb. (2) Auto-login to get JWT. (3) POST to Patient.API to create PatientRecord. If step 3 fails: `TryDeleteUserAsync()` rolls back step 1. If step 2 fails: `TryDeleteUserAsync()` rolls back step 1. No distributed transaction coordinator — purely application-level with try/catch compensation.

Q74.

What is a "soft delete" and where does your project use it? **A:** Instead of `DELETE FROM Users WHERE UserID = @id`, a soft delete sets `IsActive = false`. The record stays in the database for audit trail, reporting, and potential reactivation. `UserRepository.SoftDeleteAsync()` sets `u.IsActive = false` and saves. Login checks `user.IsActive` — blocked users can't log in but their records are preserved. Hard deletes are used for entities like Protocol (admin-only) but Users are never truly deleted.

Q75.

What is the `[AllowAnonymous]` attribute and why do only two actions in your project have it? **A:** Bypasses the globally-registered `JwtAuthFilter` — the endpoint is accessible without a valid token. Only `AuthController.Login()` and `AuthController.RegisterPatient()` are anonymous because these are the entry points where tokens don't exist yet. All other 40+ endpoints require authentication. Without `[AllowAnonymous]` on login, you'd need a token to log in — impossible.

Q76.

What is the difference between Authentication and Authorization? **A:** **Authentication** — "Who are you?" Verifies the user's identity (JWT signature check, expiry, issuer). Done by `JwtAuthFilter`. **Authorization** — "What can you do?" Checks if the authenticated user has permission for this action. Done by `RoleAuthorizeFilter`. Auth comes first — if you're not authenticated, you get 401. Then authz — if you're authenticated but wrong role, you get 403.

4.3 CORS

Q77.

What is CORS and why is it only configured in the Gateway? **A:** Cross-Origin Resource Sharing — a browser security

mechanism that blocks JavaScript from calling a different origin (protocol+host+port). Angular at `localhost:4200` calling `localhost:5046` is cross-origin. The browser first sends a preflight `OPTIONS` request. The server's response must include `Access-Control-Allow-Origin: http://localhost:4200` for the browser to proceed. Gateway is the only public-facing endpoint — all downstream services are called server-to-server (Ocelot→service), which is not subject to CORS. Configuring CORS on all 8 services would be redundant.

Q78.

What does `AllowCredentials()` do in CORS policy? When is it required? **A:** Allows cookies and HTTP authentication headers (including `Authorization: Bearer`) to be sent cross-origin. Required when your Angular frontend sends the JWT in the `Authorization` header — which is exactly your case. Without `AllowCredentials()`, the browser would strip the header from cross-origin requests. When using `AllowCredentials()`, you cannot use `.AllowAnyOrigin()` — you must list specific origins (`WithOrigins(...)`), which is why your gateway lists each Angular port explicitly.

Topic 5: Dependency Injection and Filters

5.1 DI Lifetimes

Q79.

What are the three DI lifetimes in .NET? Which did you use for what and why? **A:** **Transient** — new instance every resolution. **Scoped** — one per HTTP request. **Singleton** — one for app lifetime. Your project: All `DbContext`, repositories, and services are **Scoped** (share one DB connection per request). `IMemoryCache` (via `AddMemoryCache()`) is **Singleton** (cache must survive across requests). `IHttpClientFactory` (via `AddHttpClient()`) is **Singleton** (socket pooling). `IOptions<JwtSettings>` (via `Configure<>()`) is **Singleton** (config is immutable). Controllers are **Transient** (framework default).

Q80.

What is the "captive dependency" problem? Give an example. **A:** When a Singleton captures a Scoped dependency, the Scoped service is effectively promoted to Singleton — it never gets disposed, its `DbContext` never closes, and state bleeds across requests. Example: if `IMemoryCache`'s implementation captured `AuthDbContext` (Scoped), the `DbContext` would live forever. .NET throws `InvalidOperationException` in Development mode when it detects this. Your project avoids it: Singletons (`IMemoryCache`, `IHttpClientFactory`) have no Scoped dependencies.

Q81.

Why is `IHttpContextAccessor` registered as Singleton if it accesses per-request data? **A:** `IHttpContextAccessor` itself is a thin wrapper — it stores a reference to the current `HttpContext` in an `AsyncLocal<HttpContext>` (thread-local storage that flows with `async/await`). The Singleton registration is safe because it doesn't hold request state directly — it reads from the ambient `AsyncLocal`. Each request has its own `HttpContext` in the `AsyncLocal`, even though the accessor is shared. In `AuthDbContext`: `_httpCtx?.HttpContext?.User?.FindFirstValue(...)` reads from the current request's context at the time of the call.

Q82.

How does the DI container resolve constructor dependencies? **A:** When the container creates an instance of `AuthService`, it reads its constructor: `AuthService(IUserRepository repo, IPasswordHasher hasher, ...)`. For each parameter, it looks up the registered service by interface type, creates/finds it according to its lifetime, and injects it. This is recursive — creating `IUserRepository` causes `AuthDbContext` to be resolved first. You never call `new AuthService(...)` in your code — the container does it.

Q83.

What is the difference between `AddScoped` when registered with and without an interface? **A:** With interface: `builder.Services.AddScoped<IUserRepository, UserRepository>()` — resolves by `IUserRepository`, returns `UserRepository`. Enables mocking and substitution. Without interface: `builder.Services.AddScoped<UserRepository>()` — resolves by concrete type only. You can only inject `UserRepository`, not an abstraction. Your project always uses interface registration — critical for testability.

5.2 Filters Deep Dive

Q84.

What are the five filter types in ASP.NET Core? Which did your project implement? **A:** **Authorization filters** (`IAsyncAuthorizationFilter`) — `JwtAuthFilter`, `RoleAuthorizeFilter` — run before everything. **Resource filters** — run before model binding; not used. **Action filters** (`IActionFilter`, `IAsyncActionFilter`) — `ValidateModelFilter` (before), `ActivityLogFilter` (after). **Exception filters** (`IExceptionHandler`) — `DomainExceptionHandler` — catches exceptions from actions. **Result filters** — wrap result execution; not used. Middleware wraps all of these.

Q85.

What is the exact execution order of your filters for a successful request vs a failed request? **A:** **Successful:** `JwtAuthFilter` → `RoleAuthorizeFilter` → `ValidateModelFilter.OnActionExecuting` → Controller action → `ActivityLogFilter.OnActionExecuted`. **401 (bad token):** `JwtAuthFilter` sets `context.Result` → pipeline short-circuits, returns immediately. **403 (wrong role):** `RoleAuthorizeFilter` sets `context.Result` → returns. **422 (bad model):** `ValidateModelFilter` sets `context.Result` → returns. **DomainException thrown:** `DomainExceptionHandler.OnException` → sets `context.Result = 400`. **Unhandled exception:** `GlobalExceptionHandler` catches it → returns 500.

Q86.

What is `context.Result` in a filter and what does setting it do? **A:** In action/authorization filters, setting `context.Result` to any `IActionResult` (like `UnauthorizedResult()`) short-circuits the pipeline — subsequent filters and the action method are **not**

called. The result is returned directly. This is how `JwtAuthFilter` blocks unauthenticated requests without ever reaching the controller.

Q87.

What is `context.ExceptionHandled = true` in `DomainExceptionHandler`? **A:** Marks the exception as handled so it doesn't propagate further up the pipeline (to `GlobalExceptionHandler`). Without it, after `DomainExceptionHandler` handles a `DomainException`, the exception would still bubble up and `GlobalExceptionHandler` would also process it — resulting in two responses being written. Setting `ExceptionHandled = true` stops the propagation.

Q88.

What is `TypeFilterAttribute` and why is it used for `RoleAuthorizeAttribute`? **A:** `TypeFilterAttribute` creates a filter instance through DI, allowing constructor injection. A plain `Attribute` cannot have services injected because attributes are created by the CLR, not the DI container. `RoleAuthorizeAttribute` extends `TypeFilterAttribute` and passes the allowed `RolesEnum[]` as `Arguments`:

```
public class RoleAuthorizeAttribute : TypeFilterAttribute {
    public RoleAuthorizeAttribute(params RolesEnum[] roles)
        : base(typeof(RoleAuthorizeFilter)) {
        Arguments = new object[] { roles };
    }
}
```

The DI container creates `RoleAuthorizeFilter(RolesEnum[] allowedRoles)` with the passed roles.

Q89.

What is `IAsyncAuthorizationFilter` vs `IAuthorizationFilter`? Why use the async version? **A:** `IAuthorizationFilter.OnAuthorization()` — synchronous. `IAsyncAuthorizationFilter.OnAuthorizationAsync()` — asynchronous. `JwtAuthFilter` implements `IAsyncAuthorizationFilter` because JWT validation may involve `await` calls (reading configuration, async token validation in some implementations). Even if the current implementation is synchronous, the async interface is future-proof and preferred for I/O-bound work.

Q90.

How do global filters differ from action-level filters in registration and execution? **A:** **Global** — registered in `Program.cs` via `o.Filters.Add<FilterName>()` — runs on every action in the service. **Action-level** — applied as attributes `[RoleAuthorize(...)]` or `[AllowAnonymous]` on specific actions/controllers. Both run in the same filter pipeline. All four global filters in your project (`JwtAuthFilter`, `ActivityLogFilter`, `DomainExceptionHandler`, `ValidateModelFilter`) run on every single action. `RoleAuthorizeFilter` only runs where its attribute is placed.

Topic 6: Microservices Architecture

6.1 Core Concepts

Q91.

What is a microservice? How does LifeTrack satisfy the microservices definition? **A:** A microservice is a small, independently deployable service focused on a single business capability, with its own data store, deployed and scaled independently. LifeTrack: 8 services each with a single responsibility (auth, protocols, patients, etc.), each with its own port and independent startup, two separate databases (GovernanceDb + ClinicalDb satisfying the "polyglot persistence" principle), each deployable independently. The Docker support (docker-compose.yml exists) further confirms the deployment model.

Q92.

What is the difference between a monolith and microservices? What was LifeTrack before the migration? **A: Monolith** — single deployable unit, all features in one codebase, one database. **Microservices** — multiple independent units. LifeTrack started as an ASP.NET Core Razor Pages monolith with one database (`LifeTrackDB`). It was then migrated to 8 microservices. The migration required splitting the single `LifeTrackDB` into two databases and separating business logic into individual services.

Q93.

What are the tradeoffs of microservices over a monolith? **A: Advantages:** Independent deployment, independent scaling, technology flexibility, fault isolation (one service down doesn't take all down), clear team ownership. **Disadvantages:** Network latency between services, distributed tracing complexity, data consistency challenges (no distributed transactions), more infrastructure (8 processes instead of 1), harder to debug (request spans multiple logs). LifeTrack is a learning project so advantages outweigh complexity concerns.

Q94.

What is the "database per service" pattern? How does your project implement it? **A:** Each service owns its data and other services cannot directly access its database. LifeTrack: Auth.API owns GovernanceDb users/roles. ProtocolSite.API owns clinicalDb protocols/sites. Patient.API owns clinicalDb patients/enrollments. When services need data from another service's domain, they store a cross-service ID reference (raw `long`) and either call the API or denormalize the needed data locally.

Q95.

What is service coupling? How did you minimize it in your project? **A:** Coupling is the degree of dependency between services. High coupling — Service A directly calls Service B for every operation (can't work independently). Low coupling — services communicate only via well-defined contracts. LifeTrack minimizes coupling by: sharing only JWT tokens for identity, using cross-service ID references instead of data duplication, not having services call each other in real-time (except one saga), using the gateway as the single entry point.

Q96.

What is the API Gateway pattern? What does Ocelot provide? **A:** An API gateway is a single entry point that routes requests to appropriate microservices. It can handle cross-cutting concerns: authentication, rate limiting, SSL termination, load balancing, request aggregation. Ocelot provides reverse proxying (routing), CORS handling, and can do JWT validation (you chose not to — validation is in each service instead). The gateway decouples Angular from knowing individual service ports.

Q97.

What is service discovery? Does your project use it? **A:** Service discovery is the mechanism by which services find each other dynamically (IP/port may change on restarts or scaling). Your project uses **static discovery** — hardcoded `localhost:port` in `ocelot.json` and `appsettings.json`. Dynamic discovery would use tools like Consul or Kubernetes service DNS. Static is fine for local development; production needs dynamic discovery.

6.2 Inter-Service Communication

Q98.

What are the two types of inter-service communication? Which does your project use? **A: Synchronous** — HTTP/REST, gRPC — caller waits for response. **Asynchronous** — message queues (RabbitMQ, Azure Service Bus) — caller sends a message and continues. Your project uses synchronous HTTP (the patient registration saga calls Patient.API via `HttpClient`). Asynchronous would be better for the saga — if Patient.API is down, the message queue holds the message

until it recovers, preventing the compensating rollback.

Q99.

What is `IHttpClientFactory` and why is it better than `new HttpClient()`? **A:** `HttpClient` internally uses sockets. `new HttpClient()` per request creates and disposes sockets faster than the OS can recycle them — causing `SocketException` under load (socket exhaustion). `IHttpClientFactory` maintains a pool of `HttpMessageHandler` objects, reusing sockets and respecting DNS TTLs. Named clients (`AddHttpClient("PatientApi", ...)`) pre-configure base address and headers. Your Auth.API uses `IHttpClientFactory.CreateClient("PatientApi")` for the saga.

Q100.

What is a distributed transaction? Why doesn't your project use one? **A:** A distributed transaction (XA/2-Phase Commit) coordinates multiple databases to either all commit or all rollback atomically. In LifeTrack's patient registration, creating User (GovernanceDb) and creating Patient (ClinicalDb) ideally would be one atomic transaction. But distributed transactions require a transaction coordinator, add latency, reduce availability, and are complex. Instead, your saga uses application-level compensation (`TryDeleteUserAsync` rolls back the User if Patient creation fails).

Q101.

What is the Saga pattern? What are its two types? **A:** A saga is a sequence of local transactions with compensating actions for failure. **Choreography** — each service publishes events and reacts to other services' events (no central coordinator). **Orchestration** — a central orchestrator tells each service what to do. Your patient registration is **orchestration** — `AuthController` directly orchestrates: create user → login → call patient → compensate on failure. No event bus involved.

6.3 Data Patterns

Q102.

What is the "shared database" anti-pattern? Does your project avoid it? **A:** The anti-pattern is multiple services sharing one database and directly joining across service tables — this tightly couples services at the data level. LifeTrack partially avoids it: GovernanceDb is shared by Auth, DocumentCompliance, and Notification; ClinicalDb is shared by ProtocolSite, Patient, AdverseEvent, and AnalyticsReport. The `AuditEntries` table is written to by multiple services using `ExcludeFromMigrations()`. Ideally, each service would have its own isolated database — the current design is a pragmatic compromise for a local development project.

Q103.

What is eventual consistency? How does it appear in your project? **A:** In distributed systems, instead of immediate consistency (all services see the same data instantly), eventual consistency means data will be consistent *eventually* after propagation. In LifeTrack: `EnrollmentService.SyncPatientStatusAsync()` updates `Patient.EnrollmentStatus` after every enrollment change. Between the enrollment update and the patient status update, there's a brief window where the patient status is stale — this is eventual consistency at a tiny scale.

Q104.

What is the CQRS pattern? Does your project use it? **A:** Command Query Responsibility Segregation — separates read (Query) and write (Command) models. Your project has a mild form: separate DTOs for create (`CreateProtocolRequest`), update (`UpdateProtocolRequest`), and response (`ProtocolResponse`). Full CQRS would use separate read and write databases and separate handler classes. You could evolve toward CQRS by introducing MediatR with command/query handlers.

Topic 7: Caching

Q105.

What is `IMemoryCache` and how is it different from distributed cache (`IDistributedCache`)? **A:** `IMemoryCache` — in-process, in-RAM cache. Each service has its own isolated cache. Fast (no network). Lost on restart. Works only for single-instance deployments. `IDistributedCache` — shared cache (Redis, SQL Server) across multiple instances of the same service. Survives restarts. Slower (network hop). Your project uses `IMemoryCache` — appropriate for local single-instance development. Production multi-instance would need Redis.

Q106.

Explain the version-bump cache invalidation pattern. Why not use wildcard removal? **A:** `IMemoryCache` has no wildcard removal — you can't call `_cache.Remove("documents:list:*")`. The version-bump pattern: a version integer stored at `"documents:list:version"` is embedded in every list cache key (`"documents:list:v3:null:Draft:null:1:50"`). On any write, `BumpVersion()` increments the counter. All old `v3` keys become unreachable — no one builds `v3` keys anymore. They expire naturally after `CacheDuration` (5 minutes). Old keys are orphaned, not removed.

Q107.

What is `CacheItemPriority.NeverRemove` and why is it used on the version key? **A:** When server memory is low, .NET's `IMemoryCache` evicts cache entries starting with lowest priority (Normal → High → NeverRemove is never evicted). If the version key were evicted, `GetVersion()` would reset to 0. Suddenly all the `v3`, `v4`, `v5` keys are orphaned forever (nothing will regenerate them properly). Setting `NeverRemove` on the version key ensures the version counter survives memory pressure events — the entire invalidation system stays coherent.

Q108.

Why is the item cache invalidated differently from the list cache? **A:** Item keys are exact and known: `"documents:item:42"`. They can be directly removed with `_cache.Remove($"documents:item:{id}")` — immediate, precise, no orphans. List keys embed multiple filter combinations — you can't enumerate all possible (`protocolId`, `status`, `category`, `page`, `pageSize`) combinations to remove them individually. The version bump efficiently orphans all of them with one `Set()` call.

Q109.

What is cache stampede (thundering herd) and how would you prevent it? **A:** Cache stampede: the cache key expires, 100 simultaneous requests all see a cache miss, all start the expensive DB query simultaneously, all try to write to cache — wasting resources. Prevention: **lock-based** (only one thread regenerates, others wait), **background refresh** (refresh before expiry), **probabilistic early expiration** (random early expiry to stagger refreshes). Your project doesn't handle stampede — for production you'd add a `SemaphoreSlim` per cache key.

Q110.

What cache duration did you use and what are the tradeoffs of longer vs shorter duration? **A:** 5 minutes (`TimeSpan.FromMinutes(5)`). **Longer duration** — fewer DB queries, better performance. **Risk** — stale data served for longer after a mutation (your version-bump mitigates this for your own writes, but another process directly editing the DB would cause stale data for the full duration). **Shorter duration** — fresher data, more DB load. 5 minutes is a pragmatic balance for clinical data that changes infrequently.

Q111.

What is the `GetOrCreate()` cache method? How does it differ from `TryGetValue` + `Set`? **A:** `GetOrCreate(key, factory)` — if key exists, returns cached value. If not, calls `factory()` to create the value, stores it in cache, and returns it — atomically in one method call. Your `GetVersion()` uses it:

```
_cache.GetOrCreate(VersionKey, e => { e.Priority = CacheItemPriority.NeverRemove; return 0; });
```

`TryGetValue` + `Set` is the manual equivalent used in `ListAsync()` because the creation (DB query) is async and `GetOrCreate` doesn't have a clean async version in older .NET. In .NET 8+ there's `GetOrCreateAsync()`.

Topic 8: REST API Design

Q112.

What is REST? What are its six constraints? **A:** Representational State Transfer — an architectural style for distributed systems. Six constraints: **Stateless** (each request contains all info; no server-side session — your JWT is stateless), **Client-Server** (Angular is client, .NET APIs are server — separated concerns), **Cacheable** (responses indicate cacheability; your APIs don't set cache headers explicitly), **Uniform Interface** (resource-based URLs, standard HTTP methods — your APIs follow this), **Layered System** (Gateway sits between Angular and services), **Code on Demand** (optional — servers can send executable code; not applicable here).

Q113.

What HTTP status codes does your project use and what does each mean? **A:** `200 OK` — successful GET or PUT with body. `201 Created` — successful POST, resource created (`CreatedAtAction`). `204 No Content` — successful operation but no body (`EnrollmentsController.Update`). `400 Bad Request` — business rule violation (`DomainException` → `DomainExceptionFilter`). `401 Unauthorized` — missing/invalid JWT (`JwtAuthFilter`). `403 Forbidden` — valid JWT but wrong role (`RoleAuthorizeFilter`), or IDOR (`UnauthorizedAccessException` in `NotificationService`). `404 Not Found` — resource not found. `422 Unprocessable Entity` — validation failure (`ValidateModelFilter`).

Q114.

What is the difference between PUT and PATCH? How does your project use PATCH? **A:** `PUT` — replace the entire resource. `PATCH` — partial update of specific fields. Your project uses `PATCH` only once: `PATCH /api/kpi-reports/{id}/review` — only `Status` and `ReviewedAt` are updated, the rest of the report is untouched. All other updates use `PUT` (full resource replacement). The review is semantically an action/transition, not a data update — `PATCH` conveys that correctly.

Q115.

What is `CreatedAtAction()` and what headers does it set? **A:** Returns HTTP 201 with: (1) the created resource in the body, and (2) `Location: /api/[resource]/[id]` header pointing to the new resource's URL. `CreatedAtAction(nameof(Get), new { id = created.DocumentID }, created)` generates the URL for the `Get` action with the new ID. REST best practice — the client immediately knows where to find the created resource.

Q116.

What is idempotency in REST? Which methods should be idempotent? **A:** An operation is idempotent if calling it multiple times produces the same result as calling it once. `GET` — idempotent (same data returned). `PUT` — idempotent (same full replace). `DELETE` — idempotent (first call deletes, subsequent calls return 404 or 200 — same state). `POST` — **not** idempotent (creates a new resource each time). `PATCH` — depends on implementation. In your `NotificationService.MarkReadAsync()`: `if (n.Status != "Read") { ... }` — idempotent (calling it multiple times on the same notification doesn't change the result after the first call).

Q117.

What is content negotiation and what content type does your project use? **A:** Content negotiation is the client and server agreeing on the format of request/response data. Client sends `Accept: application/json`, server sends `Content-Type: application/json`. Your APIs only support JSON (no XML, no protobuf). `AddControllers().AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()))` configures JSON serialization with string enum names.

Q118.

What is a sub-resource route? Where does your project use it? **A:** A route that represents an action on a sub-resource rather than a top-level CRUD operation. `PATCH /api/kpi-reports/{id}/review` is a sub-resource action route — `review` is the action, `{id}` is the resource. Other examples: `/api/users/{id}/deactivate`, `/api/notifications/{id}/read`. Used when the operation doesn't fit neatly into PUT/DELETE semantics.

Q119.

What is API versioning? Does your project implement it? **A:** API versioning allows breaking changes without breaking existing clients — v1 API still works while v2 adds new features. Common approaches: URL versioning (`/api/v1/patients`), header versioning (`Api-Version: 1`), query string (`?version=1`). Your project doesn't implement versioning — all routes are unversioned (`/api/patients`). Swagger is configured with `"v1"` but this is documentation only. Production APIs should version from day one.

Topic 9: Architecture and Design Patterns

9.1 Repository and Service Pattern

Q120.

What is the Repository pattern? Why do you use it instead of injecting `DbContext` into controllers? **A:** Repository wraps data access behind an abstraction (`IUserRepository`). Benefits: (1) Controllers/services are decoupled from EF Core — you could swap SQL Server for MongoDB by changing only the repository. (2) Testable — in unit tests, inject a mock `IUserRepository` instead of a real `DbContext`. (3) Single Responsibility — controllers handle HTTP, services handle business logic, repositories handle data access. Without it, business logic leaks into controllers and data access leaks into services.

Q121.

What is the Service layer pattern? What's the difference between the controller calling the repository directly vs using a service? **A:** Service layer enforces business rules and coordinates multiple repositories. In `SiteProtocolService.CreateAsync()` — the five-step validation chain checks site existence, site status, protocol existence, protocol status, and duplicates. This logic doesn't belong in the controller (HTTP concern) or repository (data concern). If the controller called the repository directly, you'd have to duplicate these checks everywhere.

Q122.

What is the Unit of Work pattern? How does EF Core's `DbContext` implement it? **A:** Unit of Work tracks all changes across multiple repositories in one transaction and commits them together. `DbContext` IS a Unit of Work — it tracks entities from all your repositories, and `SaveChangesAsync()` writes all changes in one atomic SQL transaction. In `AuthDbContext.SaveChangesAsync()`, the `AuditEntry` rows AND the original entity changes are committed in one transaction.

Q123.

What is the interface segregation principle? How do your repository interfaces follow it? **A:** ISP (the "I" in SOLID) — don't force clients to depend on methods they don't use; split large interfaces into smaller, focused ones. Your `IUserRepository` has exactly the methods needed for user management. `IAuditLogRepository` has exactly what audit logging needs. They're separate rather than one giant `IDataRepository` with 20 methods. Services inject only the interfaces they need.

9.2 SOLID Principles

Q124.

Explain the Single Responsibility Principle with examples from your project. **A:** Every class has one reason to change. `BCryptPasswordHasher` — only handles password hashing. `JwtTokenService` — only generates JWT tokens. `DomainExceptionFilter` — only converts exceptions to HTTP responses. `UserRepository` — only handles User data access. `AuthService` — only coordinates authentication logic. Contrast: if `AuthController` contained password hashing logic, it would have two reasons to change (HTTP contract AND password algorithm).

Q125.

Explain the Open/Closed Principle with examples from your project. **A:** Open for extension, closed for modification. `Shared.CL` filters are closed for modification — you don't change `JwtAuthFilter` when adding a new service. You extend the system by adding a new service that uses the existing filter. The `DomainException` / `DomainExceptionFilter` pair is open for extension — throw new `DomainException` subtypes without modifying the filter. If you add a new microservice, you implement `IPatientRepository`, `IPatientService` without modifying existing interfaces.

Q126.

Explain the Dependency Inversion Principle with a project example. **A:** High-level modules (`AuthService`) should not depend on low-level modules (`UserRepository`). Both should depend on abstractions (`IUserRepository`). `AuthService` constructor: `AuthService(IUserRepository repo, ...)`. The `repo` is an abstraction. At runtime the DI container provides `UserRepository`. In tests, it provides `MockUserRepository`. `AuthService` doesn't know or care — it uses `IUserRepository` exclusively.

Q127.

What design pattern does the `Shared.CL` library represent? **A: Cross-Cutting Concerns** pattern — a strategy for handling aspects that span the entire application (logging, authentication, error handling, validation) in one place. Technically this is implemented via the **Chain of Responsibility** pattern (filter pipeline) and **Decorator** pattern (middleware wrapping the entire pipeline). The library itself is a **Facade** over complex security and error handling logic.

9.3 GoF Design Patterns in Your Project

Q128.

Where is the Strategy pattern used in your project? **A:** `IPasswordHasher` interface with `BCryptPasswordHasher` implementation is a Strategy. The algorithm for hashing passwords is a "strategy" that can be swapped. If you switch from BCrypt to Argon2 (newer, stronger), you implement `Argon2PasswordHasher : IPasswordHasher` and change one DI registration — no other code changes.

Q129.

Where is the Factory pattern used? **A:** `IHttpClientFactory.CreateClient("PatientApi")` is the Factory pattern — creates `HttpClient` objects without exposing creation details (socket pooling, handler recycling). The `ApiResponse<T>.Ok()` and `ApiResponse<T>.Fail()` static factory methods are also factory methods — they create the response object with specific states.

Q130.

Where is the Decorator pattern used? **A:** `GlobalExceptionHandler` decorates the entire request pipeline — it wraps `_next(context)` with try/catch, adding exception handling behavior without modifying the pipeline's core behavior. `ActivityLogFilter` decorates action execution — wraps `await next()` with logging.

Q131.

Where is the Observer pattern used or where could it be added? **A:** Not explicitly implemented in your backend (you don't use events/delegates for communication). But the Notification system conceptually implements Observer intent — when an adverse event is created (subject changes state), a notification is sent to relevant users (observers are notified). In a proper event-driven implementation, `AdverseEventService.CreateAsync()` would publish an `AdverseEventCreated` event that `NotificationService` subscribes to.

Q132.

What is the Saga pattern and where does your project use it? **A:** A saga is a sequence of steps with compensating actions for rollback (covered in Q101). The patient registration in `AuthController.RegisterPatient()` is a manual orchestration saga — three steps with compensating `TryDeleteUserAsync()` rollback. The pattern ensures data consistency across services without distributed transactions.

9.4 Domain-Driven Design Concepts

Q133.

What is a Domain Exception and why is it better than returning error codes? **A:** `DomainException` is a typed exception for business rule violations. Compared to returning `{ success: false, errorCode: "PROTOCOL_HAS_ASSIGNMENTS" }` from every service method: the caller doesn't need to check return values — if the business rule passes, the method returns normally; if it fails, execution immediately jumps to the catch block (`DomainExceptionFilter`). The code reads as sequential happy-path logic. No null checks, no conditional chains after every service call.

Q134.

What is the difference between a domain entity and a DTO? **A:** **Domain entity** (`Protocol`, `Patient`, `User`) — maps directly to a DB table, tracked by EF Core's change tracker, contains all fields including sensitive ones. **DTO** (`ProtocolResponse`, `CreatePatientRequest`) — a data transfer object designed for the API contract. Response DTOs may omit sensitive fields (e.g., `User` has `PasswordHash` but `UserResponse` doesn't). Request DTOs contain only what the caller provides. Separation ensures internal model changes don't break the API contract.

Q135.

What is an Aggregate Root in DDD? Can you identify one in your project? **A:** An aggregate root is the main entity in a cluster of related objects. External code can only access the cluster through the root. In your project: `Patient` is an aggregate root for the `Patient → Enrollment → Visit` cluster. You don't add visits directly — you go through `PatientService → EnrollmentService → VisitService`. The constraint "cannot add visits to a withdrawn enrollment" enforces aggregate invariants.

Topic 10: SQL Server and Database Design

10.1 SQL Server Concepts

Q136.

What is the difference between `nvarchar(200)` and `varchar(200)` in SQL Server? **A:** `nvarchar` stores Unicode characters (2 bytes per char) — supports any language, symbols, emojis. `varchar` stores ASCII/extended ASCII (1 byte per char) — only Western characters. EF Core maps `string` to `nvarchar` by default with `[MaxLength(n)] → nvchar(n)`. Your project uses `nvarchar` everywhere (`nvarchar(200)`, `nvarchar(50)`, etc.) — correct for a healthcare system that may have international names.

Q137.

What is the difference between `datetime2` and `datetime` in SQL Server? **A:** `datetime2` — higher precision (0-7 decimal places), wider range (0001-9999), smaller storage (6-8 bytes). `datetime` — lower precision (3.33ms), narrower range (1753-9999), 8 bytes. EF Core maps `Dateime` to `datetime2` by default. All your audit timestamps, dates are `datetime2`.

Q138.

What are SQL Server indexes? What types of indexes did you create? **A:** An index is a data structure (B-tree) that enables fast lookups. Without an index, SQL Server scans all rows (table scan $O(n)$). With an index, it seeks to matching rows (index seek, typically $O(\log n)$). **Non-clustered indexes** — your `HasIndex()` calls create these. **Filtered index** — `HasIndex(p => p.UserID).IsUnique().HasFilter("[UserID] IS NOT NULL")` only indexes non-null values. **Composite index** — `HasIndex(n => new { n.UserID, n.Status })` in `Notification.API` indexes on two columns together.

Q139.

When is a composite index more efficient than two separate indexes? **A:** When the query filters on BOTH columns together. `WHERE UserID = 42 AND Status = 'Unread'` with separate indexes: SQL Server must intersect two index results. With composite `(UserID, Status)` index: SQL Server seeks directly to rows matching both — one operation. The composite index is also more efficient for the "bell count" query (most common notification query) than two individual indexes.

Q140.

What is a SQL transaction? How does EF Core's `SaveChangesAsync()` use transactions? **A:** A transaction is a unit of work that is either fully committed or fully rolled back. `SaveChangesAsync()` automatically wraps all changes in a transaction. In `AuthDbContext.SaveChangesAsync()`, the original entity changes AND the new `AuditEntry` rows are committed together — if any INSERT fails, none are committed. You can explicitly use `using var tx = await _db.Database.BeginTransactionAsync()` for multi-step operations.

Q141.

What is the difference between `RESTRICT` and `CASCADE` in SQL foreign keys? **A:** `RESTRICT` — prevents deleting the parent if any child rows exist (error is thrown). `CASCADE` — deleting the parent automatically deletes all child rows. Your `Patient.API` migration creates `FK_Enrollments_Patients_PatientID` with `ReferentialAction.Restrict` (can't delete a patient with enrollments) and `FK_Visits_Enrollments_EnrollmentID` with `ReferentialAction.Cascade` (deleting enrollment deletes its visits).

Q142.

What is SQL Server's `OFFSET/FETCH` and how does EF Core pagination use it? **A:** The SQL Server syntax for pagination: `ORDER BY col OFFSET x ROWS FETCH NEXT y ROWS ONLY`. `Skip((page-1) * pageSize)` translates to `OFFSET ((page-1)*pageSize) ROWS`. `Take(pageSize)` translates to `FETCH NEXT pageSize ROWS ONLY`. Requires `ORDER BY` — without it, SQL Server doesn't guarantee row order and pagination is meaningless.

Q143.

What is the `sys.columns` system table? How did you use it in migrations? **A:** `sys.columns` is a SQL Server catalog view containing metadata about all columns in all tables. In your defensive migrations:

```
IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('KPIReports') AND name = 'Status')
    ALTER TABLE [KPIReports] ADD [Status] nvarchar(20) NOT NULL DEFAULT 'Draft';
```

`OBJECT_ID('KPIReports')` returns the table's internal ID. Checking `sys.columns` before `ALTER TABLE ADD` makes the migration idempotent.

Q144.

What is an identity column in SQL Server? What does `.Annotation("SqlServer:Identity", "1, 1")` mean? **A:** An identity column auto-increments. `(1, 1)` means start at 1, increment by 1. Every `[Key]` long property in EF Core becomes `bigint IDENTITY(1, 1)` — SQL Server assigns the next value on INSERT. After `INSERT`, EF Core reads the generated ID with `SELECT SCOPE_IDENTITY()` and updates the entity's PK property — which is why `created.EventID` is populated after `_repo.AddAsync(ae)`.

10.2 Database Design

Q145.

What is database normalization? What normal form is your project's schema? **A:** Normalization eliminates data redundancy and update anomalies. Your schema is generally in **3NF** (Third Normal Form): (1NF) — atomic values, no repeating groups. (2NF) — no partial dependencies on composite keys. (3NF) — no transitive dependencies (no non-key column depending on another non-key column). `KpiReport` is a wide flat table (violates normalization slightly for reporting purposes — denormalized intentionally for query performance).

Q146.

Why does ProtocolSite.API own the `AuditEntries` table in ClinicalDb? **A:** ProtocolSite.API was the first service created for ClinicalDb and ran `InitialCreate` migration which created `AuditEntries`. Since EF Core migrations are cumulative and the table is already in the DB, all subsequent services (AdverseEvent, Patient, AnalyticsReport) use `ExcludeFromMigrations()` to avoid duplicating the CREATE. The decision was pragmatic — one service must own and migrate it; ProtocolSite was the logical choice as the first ClinicalDb service.

Q147.

What is the purpose of having two separate databases (GovernanceDb and ClinicalDb)? **A:** **Separation of concerns** — governance data (users, roles, documents, notifications) is separate from operational data (protocols, patients, trials). **Security** — different database permissions; clinical staff may not need access to governance tables. **Performance** — clinical data (high-volume trial data) doesn't compete with governance queries. **Compliance** — regulatory frameworks may require separating patient data from administrative data. **Scalability** — scale ClinicalDb independently from GovernanceDb.

Q148.

What is a junction table? How does `SiteProtocol` work as one? **A:** A junction table (also called a bridge or join table) resolves a many-to-many relationship by storing FK pairs plus additional attributes. `SiteProtocol` links `Protocol` and `Site` (many-to-many) but also adds payload: `InvestigatorID`, `Phase`, `Status`, `InitiationDate`. A pure junction table has only the two FKs. `SiteProtocol` is a "rich" junction table because the relationship itself has meaningful attributes.

Q149.

What is the difference between `sp_rename` and `ALTER TABLE RENAME COLUMN`? Why did DocumentCompliance.API have issues with it? **A:** In older SQL Server, renaming a column requires `EXEC sp_rename 'Table.OldColumn', 'NewColumn', 'COLUMN'`. `ALTER TABLE ... RENAME COLUMN` was added in SQL Server 2016+. DocumentCompliance.API's `InitialCreate` migration had column named `Type`. The rename to `Category` was done by manually changing the model without creating a proper migration — causing the snapshot to show `Type` while the actual model showed `Category`. The `PendingModelChangesWarning` was suppressed instead of properly creating a rename migration.

Topic 11: Configuration, Swagger, and Tooling

Q150.

What is `appsettings.json` vs `appsettings.Development.json`? How are they loaded? **A:** `appsettings.json` — base configuration for all environments. `appsettings.Development.json` — overrides for Development environment only. ASP.NET Core loads both and overlays: environment-specific wins over base. `ASPNETCORE_ENVIRONMENT=Development` controls which environment file is loaded. In `launchSettings.json`, `"ASPNETCORE_ENVIRONMENT": "Development"` sets this for local runs. Your `appsettings.Development.json` files just set log level to `Information`.

Q151.

What is `IConfiguration` and how do you access nested config values? **A:** `IConfiguration` reads from all configured sources (JSON files, env vars, command line). Access: `_config["Jwt:Secret"]` for nested values (`:` is the section separator). `_config.GetSection("Jwt").Get<JwtSettings>()` deserializes a section. `Configuration.GetConnectionString("ClinicalDb")` is shorthand for `Configuration["ConnectionStrings:ClinicalDb"]`.

Q152.

What is the Options pattern (`IOptions<T>`)? How did you use it for `JwtSettings`? **A:** `IOptions<T>` provides strongly-typed access to a configuration section. Registration: `builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("Jwt"))`. Injection: `JwtTokenService(IOptions<JwtSettings> opts)`. Access: `opts.Value.Secret`. Compared to `IConfiguration["Jwt:Secret"]`: type-safe, auto-bound from config, easier to test (mock `IOptions<JwtSettings>`). `IOptions<T>` is Singleton — config is read once. `IOptionsSnapshot<T>` reloads per request. `IOptionsMonitor<T>` watches for changes.

Q153.

What is Swagger/OpenAPI? How is it configured in your project? **A:** Swagger is a specification for documenting REST APIs. OpenAPI is the standard the spec follows. In your project: `AddSwaggerGen()` generates the OpenAPI JSON spec. `UseSwagger()` serves it at `/swagger/v1/swagger.json`. `UseSwaggerUI()` serves the interactive HTML UI at `/swagger`. `AddSecurityDefinition("Bearer", ...)` adds the padlock button. `AddSecurityRequirement(...)` marks all endpoints as requiring JWT in Swagger. Only enabled in `IsDevelopment()`.

Q154.

What is `launchSettings.json`? How does it control the gateway port conflict with the Angular dev server? **A:** `launchSettings.json` defines launch profiles (http, https, Docker) with `applicationUrl` specifying the port. The Gateway is on 5046, all services on their own ports. The Angular dev server conflict: Angular's `ng serve` tries 4200 first; if occupied, picks a random port like 62536. This random port then appears in CORS errors. Fix: set `"port": 4200` in Angular's `angular.json` to always use 4200, then simplify the Gateway CORS list.

Q155.

What is `docker-compose.yml`? What does it enable for your project? **A:** Docker Compose defines and runs multi-container applications. For LifeTrack, it would define all 8 services, their ports, environment variables (connection strings, JWT secrets), network connections, and potentially a SQL Server container. With `docker compose up`, all 8 services start in containers with proper networking. Currently your project has the file but services run locally without Docker in development.

Topic 12: Testing Concepts

Q156.

What is the difference between unit tests, integration tests, and end-to-end tests? **A: Unit tests** — test one class in isolation, mock all dependencies. Fast, many, no I/O. Test `AuthService.LoginAsync()` with a mocked `IUserRepository` that returns a predefined user. **Integration tests** — test multiple layers together, may use real DB (in-memory). Test that `POST /api/auth/login` → `AuthService` → `UserRepository` → DB returns a valid JWT. **E2E tests** — test the full stack including the UI. Slow, few, fragile. The conversation mentioned setting up `WebApplicationFactory<Program>` for integration testing `IdentityService`.

Q157.

What is `WebApplicationFactory<TProgram>` in ASP.NET Core testing? **A:** Creates an in-memory test server running your actual ASP.NET Core app. Used for integration tests: `HttpClient client = factory.CreateClient()`. You can override DI registrations — replace real `DbContext` with in-memory EF Core: `services.RemoveAll<DbContextOptions<AuthDbContext>>(); services.AddDbContext<AuthDbContext>(o => o.UseInMemoryDatabase("TestDb"))`. Requires `public partial class Program {}` in `Program.cs` to expose the type.

Q158.

Why does the integration test setup use `RemoveAll<T>()` instead of `Remove<T>()`? **A:** `IServiceCollection.Remove(descriptor)` removes one specific descriptor. `RemoveAll<T>()` removes ALL registrations of type `T`. EF Core may register multiple related descriptors for `DbContextOptions<T>` (for pooling, validation, etc.). Using `Remove()` might only remove one and leave orphaned descriptors that still reference the real SQL Server. `RemoveAll<T>()` ensures a clean slate before adding the in-memory DB.

Q159.

What is mocking and which library would you use with your project? **A:** Mocking creates fake implementations of interfaces for testing. Instead of hitting the real database, a mock `IUserRepository` returns predetermined data. Popular libraries: **Moq** (`Mock<IUserRepository>().Setup(r => r.GetByEmailAsync("test@test.com")).ReturnsAsync(testUser)`), **NSubstitute**, **FakeItEasy**. Your interfaces (`IUserRepository`, `IAuthService`) are perfectly designed for mocking — all dependencies are injected abstractions.

Q160.

What is the Arrange-Act-Assert pattern in unit tests? **A:** Standard unit test structure. **Arrange** — set up the test: create mocks, configure mock return values, instantiate the class under test. **Act** — call the method being tested. **Assert** — verify the result: check return values, verify mock method calls, assert exceptions. Example testing `AuthService.LoginAsync` with wrong password: Arrange (mock returns a user with a hash), Act (call `LoginAsync` with wrong password), Assert (`AuthException` is thrown with expected message).

Topic 13: Advanced .NET and C# Topics

Q161.

What is `IHttpContextAccessor`? Why is it needed in `AuthDbContext`? **A:** Provides access to the current `HttpContext` from non-controller code. Controllers have direct access via `this.HttpContext`. But `AuthDbContext` is a service — it has no direct access to `HttpContext`. `IHttpContextAccessor` bridges this: it stores the current `HttpContext` in `AsyncLocal<>` (thread-local that flows with `async`), allowing any service to read the current request's context. Used in `AuthDbContext` to read `User.FindFirstValue(ClaimTypes.NameIdentifier)` for `ChangedByUserID` in audit rows.

Q162.

What is `JsonStringEnumConverter` and where is it registered in your project? **A:** By default, `System.Text.Json` serializes enums as integers (3). `JsonStringEnumConverter` makes them serialize as their string name ("Investigator"). Registered in all services:

```
.AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()))
```

This means responses return `"status": "Active"` instead of `"status": 1` — human-readable and decoupled from enum integer values.

Q163.

What is `static readonly` vs `const`? Give an example from your project. **A:** `const` — compile-time constant, must be a primitive type, inlined at compile time. `static readonly` — runtime constant, can be any type, initialized once at class loading. In your services: `private static readonly TimeSpan CacheDuration = TimeSpan.FromMinutes(5)` and `private static readonly JsonSerializerOptions _jsonOptions = new() { ... }`. These can't be `const` because they're non-primitive types. `static readonly` ensures the expensive object is created once and shared.

Q164.

What is the `params` keyword? Where does your project use it? **A:** `params` allows a method to accept a variable number of arguments as an array, called without explicitly creating the array. In `RoleAuthorizeAttribute`:

```
public RoleAuthorizeAttribute(params RolesEnum[] roles)
```

Usage: `[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager, RolesEnum.Investigator)]`. Without `params`, you'd need `[RoleAuthorize(new[] { RolesEnum.Admin, RolesEnum.CTM })]` — more verbose.

Q165.

What is the difference between `Task.FromResult()`, `Task.CompletedTask`, and `async` method returning a value? **A:** `Task.FromResult(value)` — creates an already-completed task with a result; avoids `async/await` overhead for synchronous code paths. `Task.CompletedTask` — already-completed `Task` (no result). A true `async` method with `async/await` suspends at each await point. In `AuditLogService.RecordAsync()`: `return _repo.AddAsync(log)` — returns the task directly without `async/await` overhead since there's nothing to do after the call.

Q166.

What is the difference between `string.IsNullOrEmpty()` and `string.IsNullOrWhiteSpace()`? **A:** `IsNullOrEmpty()` returns true for null or "". `IsNullOrWhiteSpace()` returns true for null, "", or strings containing only spaces/tabs/newlines. Your project uses both: `IsNullOrEmpty()` for simple null checks on filter parameters. `IsNullOrWhiteSpace()` in `DocumentService` for `DocumentLink` sanitization — "" (just spaces) should be treated as no link. Security-conscious APIs always prefer `IsNullOrWhiteSpace`.

Q167.

What is `StringComparer.OrdinalIgnoreCase` and when do you use it? **A:** A string comparer that ignores case using ordinal (byte-by-byte) comparison — fast and culturally invariant. Used in `DerivePatientStatus()`:

```
.ToHashSet(StringComparer.OrdinalIgnoreCase)
```

So "active", "Active", "ACTIVE" are all treated as equal. Without it, `"active" == "Active"` is false. Use `OrdinalIgnoreCase` for case-insensitive comparisons on data (status values, role names). Use `Ordinal` for case-sensitive exact matching (tokens, keys).

Q168.

What is `DateTimeKind` and why does `AuditLogService` use `DateTime.SpecifyKind()`? **A:** `DateTimeKind.Unspecified` (default from SQL Server), `DateTimeKind.Local` (local time zone), `DateTimeKind.Utc` (UTC). SQL Server's `datetime2` stores no timezone info — returns with `Unspecified`. The JSON serializer sees `Unspecified` and doesn't append "Z" suffix, so Angular treats it as local

time → incorrect display. `DateTime.SpecifyKind(l.ActionTime, DateTimeKind.Utc)` tells the serializer "this IS UTC" → serializes as `"2025-05-25T10:02:08Z"` → Angular displays correctly.

Q169.

What is `Encoding.UTF8.GetBytes()` and why is it used for the JWT secret? **A:** `JwtSecurityTokenHandler` needs the secret as a `SymmetricSecurityKey` which takes `byte[]`. The JWT secret in `appsettings.json` is a string (`"ThisistheJWTKEY..."`). `Encoding.UTF8.GetBytes(secret)` converts the string to its UTF-8 byte representation. The key length matters — HMAC-SHA256 needs at least 256 bits (32 bytes). Your 46-character ASCII secret = 46 UTF-8 bytes = 368 bits — sufficient.

Q170.

What is `Guid.NewGuid()` and how is it used for the JWT `jti` claim? **A:** `Guid.NewGuid()` generates a globally unique 128-bit identifier, formatted as `"xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"`. Used as the `jti` (JWT ID) claim — each issued token has a unique ID. This enables token blacklisting if implemented: store used `jti` values in a blacklist and reject tokens whose `jti` is in the list. Currently you don't implement a blacklist, but the `jti` is there for future use.

Topic 14: Project-Specific Q&A

Q171.

Why does ProtocolSite.API have two migrations while most services have one? **A:** `InitialCreate` created the initial schema with `Phase` on the `Protocols` table. `MovePhaseToSiteProtocol` moved `Phase` to `SiteProtocols` — a design correction recognizing that phase belongs to a specific site's execution of a protocol, not to the protocol globally. AnalyticsReport.API has four migrations showing incremental feature growth (base → KPI fields → review workflow → date ranges).

Q172.

What is the `PendingModelChangesWarning` and why is it suppressed in DocumentCompliance.API? **A:** EF Core emits this warning when the current `DbContext.OnModelCreating()` configuration doesn't match the `ModelSnapshot.cs`. In DocumentCompliance.API: the model was changed (Type→Category, ProtocolID made nullable, FileName/FilePath added to DB but not model) without running `dotnet ef migrations add`. The snapshot reflects the old state, the model is newer. The correct fix is `dotnet ef migrations add SyncModelChanges` to create a migration capturing all pending changes. Suppression is a technical debt workaround.

Q173.

Why does Patient.API's `.csproj` reference `BCrypt.Net-Next` even though BCrypt isn't used in Patient.API? **A:** This is a leftover from when the project was merged (Hari's features merged into Naveen's base). Naveen's Patient.API codebase may have initially included BCrypt for some planned feature, or it was copied from Auth.API's dependencies without cleanup. The reference is harmless but wasteful — it adds to build time and deployment size. A cleanup PR would remove it.

Q174.

What is the difference between `AuthDbContext` using `IHttpContextAccessor` for `CurrentUserID` vs `AnalyticsDbContext` using a public property? **A:** `AuthDbContext` injects `IHttpContextAccessor` and reads `ClaimTypes.NameIdentifier` automatically in every `SaveChangesAsync()` call — the audit trail is always correct as long as there's an active HTTP request. `AnalyticsDbContext` exposes `public long? CurrentUserID { get; set; }` — the calling service must set it manually before calling `SaveChangesAsync()`. If the service forgets (as `KpiReportService` does), audit entries have `ChangedByUserID = null`. The `IHttpContextAccessor` approach is more robust and automatic.

Q175.

Why does `EnrollmentService.SyncPatientStatusAsync()` invalidate the patient cache even though it's in a different service? **A:** Within Patient.API, all services (`PatientService`, `EnrollmentService`, `VisitService`) share the **same singleton `IMemoryCache` instance** — they're all in the same process. When `EnrollmentService` updates a patient's status via `UpdateEnrollmentStatusAsync()`, it also calls `_cache.Remove($"patients:item:{patientId}")` and `BumpVersion("patients:list:version")` — invalidating the patient cache entries managed by `PatientService`. This cross-service cache coordination within one process is possible because they share the same in-memory cache instance.

Q176.

What was the OneDrive path issue during development and how was it solved? **A:** The LifeTrack project was located in a OneDrive-synced corporate path (`C:\Users\247xxx\...`). OneDrive continuously syncs file changes. During .NET builds: (1) OneDrive creates `obj(1)` duplicate folders conflicting with `obj`. (2) `.csproj` files get reverted to older versions when OneDrive resolves sync conflicts. (3) NuGet source generator DLLs in `obj` get file-locked by OneDrive's sync process. Solution: move project to `C:\dev\LifeTrack` using `robocopy` with `/XD obj bin` to exclude build artifacts — a path outside OneDrive sync scope.

Q177.

Why does each service have its own `JwtSettings.cs` tombstone file even after centralizing to `Shared.CL`? **A:** Documentation and communication. Before `Shared.CL`, each service had its own `JwtSettings` class for reading JWT config. After centralization, the local classes were removed — but instead of deleting the files, they were kept as comment-only shells: "This file has been intentionally removed. JWT validation is now handled by `Shared.CL.Filters.JwtAuthFilter`." This prevents confusion for future developers who might wonder why there's no local JWT class.

Q178.

How would you add a new microservice to LifeTrack? What steps would you follow? **A:** (1) Create new ASP.NET Core Web API project. (2) Add `Shared.CL` project reference. (3) Copy `appsettings.json` pattern (same JWT secret, correct DB connection string). (4) Register the four global filters in `Program.cs`. (5) Call `app.UseGlobalExceptionHandler()` first. (6) Create `DbContext` with auto-audit `SaveChangesAsync`. (7) Define models, repositories (with interfaces), services (with interfaces). (8) Add routes to `ocelot.json` in Gateway.API. (9) Register DI in `Program.cs`. (10) Run `dotnet ef migrations add InitialCreate` and `dotnet ef database update`.

Q179.

What monitoring or observability would you add to this project in a production setting? **A:** (1) **Structured logging** — replace `Console.WriteLine` with Serilog/NLog writing to Elasticsearch or Azure Monitor. (2) **Distributed tracing** — OpenTelemetry to trace a request across all 8 services using the `TraceId` from `ErrorDetail`. (3) **Health checks** — `AddHealthChecks().AddDbContext<AuthDbContext>()` + `/health` endpoint for each service. (4) **Metrics** — Prometheus + Grafana for request rates, error rates, cache hit ratios. (5) **Alerting** — Alert when error rate or response time spikes. (6) **Correlation IDs** — propagate `X-Correlation-Id` header through Ocelot to all downstream services.

Q180.

What improvements would you make to the LifeTrack backend architecture? **A:** (1) **Asymmetric JWT** — use RSA keys: Auth.API holds the private key, services validate with the public key — more secure. (2) **Refresh tokens** — 15-minute access tokens + long-lived refresh tokens stored in the DB. (3) **Event-driven notifications** — publish domain events (adverse event created) via a message bus (RabbitMQ/Azure Service Bus) instead of direct notification HTTP calls. (4) **Redis cache** — replace `IMemoryCache` with Redis for multi-instance scalability. (5) **API versioning** — add `/v1/` prefixes before breaking changes. (6) **Rate limiting** — protect login endpoint from brute force. (7) **Proper HTTPS** — enforce HTTPS in production. (8) **Isolated databases** — give each service its own DB instead of sharing GovernanceDb and ClinicalDb.